Milestone 3 Report

Studying Rhino: Results, Comparison with Baselines and What Could Be Reproduced

CSE 516 - Probabilistic Graphical Models, Monsoon Semester 2026
Group: g12_th_conditional-ind-graphs_ind | Track: Theoretical AI (Individual)
Student: Het Patel, AU2440266

1. Problem Statement and Research Question

The problem is temporal causal discovery. I observe D variables over time. I want to know which variable causes which, at which time lag, and whether a cause acts inside the same time step. The second case is called an instantaneous effect. The base paper is Rhino [1]. It learns a graph over the variables and their lags. It also models the noise so that it can change with the past values of the variables. This is called history-dependent noise.

Rhino assumes that the causal structure does not change over time and that no hidden common causes exist. It also assumes that the model is minimal and that the joint density is well defined [1, Appendix B]. The graph of the current time step must be acyclic. These assumptions hold in every experiment of this report.

The research question of the project is this:

When does Rhino recover the temporal causal graph better than other methods, and which of its parts explain the gain? The parts are the instantaneous effects, the history-dependent noise, the model lag and the training length.

I state four hypotheses from the claims of the paper [1]. H1: Rhino is better than the baselines when the noise is history-dependent. H2: it has no clear advantage when the noise is not history-dependent. H3: removing the instantaneous effects hurts when the data contain them. H4: a model lag below the true lag hurts, and a model lag above it does not.

M3 asks what the evidence says. I ran the experiments of the paper with the authors' released code. All methods saw exactly the same data. I added paired tests, a precision and recall analysis, and a study of treatment effect error. M1 gave the problem and the review of existing work. M2 gave the model and the setup. This report restates only what M3 needs, so it can be read alone.

2. Existing Work and SOTA Anchor (Revisited)

The methods near Rhino differ in what they can model. Table 1 shows this, as described in the paper [1, Section 5 and Appendix C]. A survey of time series methods is in [12].

Table 1. What each method can model

Method	Type	Nonlinear	Instantaneous	History-dependent noise
VAR-LiNGAM [2]	Linear VAR, non-Gaussian noise	No	Yes	No
DYNOTEARS [3]	Linear VAR, continuous DAG penalty	No	Yes	No
PCMCI+ [4]	Constraint-based, conditional independence tests	Depends on the test (partial correlation here)	Yes, up to an equivalence class	Not modelled
cMLP, cLSTM [6], TCDF [7], SRU, eSRU [5]	Granger causality with neural networks	Yes	No	No
Rhino [1]	Neural VAR with a conditional spline flow	Yes	Yes	Yes

The base paper compares Rhino with exactly these methods. I keep them as my SOTA anchor. On synthetic data the strongest comparator in the paper is PCMCI+. On real data it is the family of nonlinear Granger methods. I did not survey work that appeared after the paper. This is a limit of this report. I revise the position in Section 10.

3. The Model: Representation, Learning and Inference

3.1 Representation

Rhino is a structural equation model over time. The value of variable i at time t is

$$X_{i,t} = f_i(Pa_i(<t), Pa_i(t)) + g_i(Pa_i(<t), \varepsilon_{i,t})$$

The sets Pa(<t) and Pa(t) are the lagged and the instantaneous parents of variable i given by the graph G. The function f is a neural network. The function g is a conditional spline flow that transforms the noise ε. It depends only on the lagged parents. So the noise can grow or shrink with the history. The graph is stored as a tensor of shape [lag + 1, D, D]. Slice 0 holds the instantaneous edges and slice τ holds the edges from t − τ to t [1, Section 3].

I use four versions of the model. **Rhino** has the conditional spline noise. **Rhino+g** has Gaussian noise that does not depend on history. **Rhino+s** has a spline noise that does not depend on history. **Rhino+NoInst** is Rhino with the instantaneous effects switched off. These are the variants of the paper, so each one tests a single idea.

3.2 Learning

Rhino learns a distribution over graphs and not one graph. The distribution q(G) is a product of independent Bernoulli variables. The parameters are found by maximising the evidence lower bound (ELBO):

$$ELBO = E_{q(G)}[\Sigma_n \log p_\theta(X^{(n)} | G) + \log p(G)] + H(q(G))$$

The prior p(G) rewards sparse graphs and penalises cycles in the instantaneous part. The cycle penalty is $h(G_0) = tr(\exp(G_0 \circ G_0)) - D$, which is zero only for a DAG [11]. Its weight grows during training in steps of an augmented Lagrangian method. The graph samples use the Gumbel-softmax estimator [1, Section 3.2]. The number of these steps is the training length. It matters in Section 7.

3.3 Inference

After training I read the graph from q(G). Two readouts are used. The edge probabilities give a ranking of all possible edges, which is scored by AUROC. Rounding at 0.5 gives one graph, which is scored by F1 and SHD. Treatment effects come from simulation. The model is rolled forward with the incoming edges of the treated variable removed, once for each graph sample. The mean over graph samples is called E[ATE]. The same simulation with the single most likely graph is called ML ATE [1, Appendix D]. This follows the design of DECI, the static model on which Rhino builds [10]. The paper proves that the graph is identifiable under conditions on f and g [1, Theorem 1]. I did not re-prove it. My experiments test what happens in practice.

4. Experimental Design

Table 2. Experiments of the paper and what I ran

Paper element	What I ran	Status
Synthetic data, Fig 1, 3, 4, 5	120 datasets of 5, 10 and 20 nodes, six methods	Done. 40 nodes not run.
Lag ablation, Table 1	Lag 1, 2 and 3 on the same 120 datasets	Done (one cell has 39 of 40)
Treatment effect, Fig 2	360 trained models (three variants on 120 datasets)	Done
DREAM3, Tables 2, 6, 7	Five networks, size 100, AUROC	Partial. See Section 7.
Netsim, Tables 3, 8, Fig 8	Sim-3, five runs per setting	Done

Data

Synthetic data. I use the authors' generator [15]. A dataset is defined by the number of nodes (5, 10, 20), the graph family (Erdős-Rényi or scale-free), instantaneous effects on or off, history-dependent noise on or off, and the data seed (0 to 4). This gives 40 datasets per dimension and 120 in total. Datasets with instantaneous effects have 4N instantaneous and 2N lagged edges. Datasets without them have 2N lagged edges. Each dataset has 50 training series of 200 steps with 50 burn-in steps. The functions are random MLPs. The true lag is 2.

Real data. DREAM3 in silico size 100 has five gene networks. Each has 46 trajectories of 21 time points and 100 genes [8, 9]. Netsim Sim-3 has 15 brain regions and 200 time steps per subject. I train on subjects 2 to 6 and validate on subjects 7 and 8, as in the paper [1, Appendix G.4]. The preprocessing steps and checksums are in Data/README.md.

Methods and settings

Table 3. Methods compared

Method	Settings	Runs per dataset
Rhino, Rhino+g, Rhino+s	Authors' code (causica v0.0.0), lag 2, sparsity weight 9 (Rhino, Rhino+s) and 5 (Rhino+g) as in the paper, CPU	1 model seed per dataset
DYNOTEARS [3]	Authors' wrapper, lag 2, default settings of the released code	1 (deterministic)
VAR-LiNGAM [2]	Lag 2, default settings with pruning	1 (deterministic)
PCMCI+ [4]	Lag 2, partial correlation test, significance level 0.05, all DAGs of the equivalence class enumerated	1
cMLP, cLSTM, TCDF, SRU, eSRU	Numbers quoted by the paper from [5]. Not re-run.	Not applicable

KPIs

Table 4. KPIs

Layer	KPI	How it is computed
Graph	Orientation F1 (instantaneous, lagged, temporal)	Edge with the correct direction and lag counts as correct. Rounded at 0.5.
Graph	Adjacency F1, precision, recall, number of predicted edges	Direction is ignored for adjacency F1.
Graph	SHD	As computed by the released code.
Graph	AUROC of the summary graph	Edge probabilities, max over lags (real data).
Fit	Training log-likelihood per sample	Last logged value of each run. It is not a held-out value.
Inference	RMSE of the average treatment effect	E[ATE] and ML ATE against 5000 true intervention samples.
Cost	Wall time per job	Minutes on one CPU worker with 2 threads.

Statistics and seeds

Every method is run on every dataset, so comparisons are paired. I report the mean and standard deviation over datasets. The 95 % interval is the t interval. For a difference between two methods I report the mean of the paired differences with a bootstrap interval of 10000 resamples. The test is the Wilcoxon signed-rank test [14] with the Holm correction [13] inside each dimension and metric. For the real data I average five runs with different model seeds and report the standard error, as the paper does. I use the Welch test for differences. The synthetic data seeds are 0 to 4. Each dataset has one model seed. So the spread in the tables is the spread over datasets and not over model seeds.

Runs were made on Windows 11 with Python 3.9 and CPU only. The torch version of the authors' code (1.12.1) has no kernels for the new GPU of the workstation. Up to 16 workers ran in parallel with 2 threads each. A job had a time limit of 10 hours.

5. Results on Synthetic Data

5.1 Rhino against the baselines

Table 5. Temporal orientation F1, mean ± standard deviation over 40 datasets per dimension

Method	5 nodes	10 nodes	20 nodes
Rhino	0.666 ± 0.139	0.554 ± 0.142	0.440 ± 0.189
Rhino+g	0.535 ± 0.112	0.465 ± 0.077	0.368 ± 0.063
Rhino+s	0.346 ± 0.259	0.312 ± 0.222	0.204 ± 0.183
DYNOTEARS	0.469 ± 0.091	0.367 ± 0.096	0.307 ± 0.117
VAR-LiNGAM	0.036 ± 0.068	0.036 ± 0.056	0.028 ± 0.037
PCMCI+	0.545 ± 0.131	0.479 ± 0.117	0.388 ± 0.120

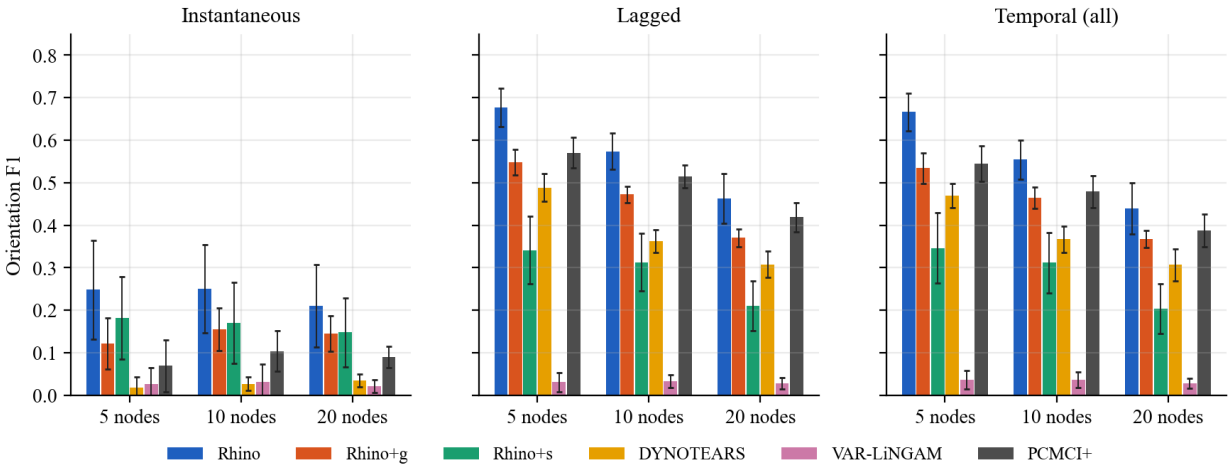


Fig. 1: Orientation F1 of the instantaneous part, the lagged part and the whole temporal graph. Bars are means over datasets with the 95 % interval. The instantaneous part uses the 20 datasets per dimension that have instantaneous effects.

Table 6. Paired difference in temporal F1, Rhino minus the other method, with the 95 % t interval (all datasets of a dimension)

Nodes	Other method	Mean difference [95 % interval]	Wins / losses	p (Holm)
5	PCMCI+	+0.122 [+0.068, +0.175]	28 / 12	<0.001
5	Rhino+g	+0.132 [+0.078, +0.186]	29 / 11	<0.001
5	DYNOTEARS	+0.197 [+0.151, +0.243]	37 / 3	<0.001
5	Rhino+s	+0.320 [+0.254, +0.386]	37 / 3	<0.001
5	VAR-LiNGAM	+0.630 [+0.581, +0.680]	40 / 0	<0.001
10	PCMCI+	+0.075 [+0.032, +0.119]	26 / 14	0.003
10	Rhino+g	+0.089 [+0.052, +0.127]	28 / 12	<0.001
10	DYNOTEARS	+0.187 [+0.137, +0.238]	35 / 5	<0.001
10	Rhino+s	+0.242 [+0.197, +0.287]	39 / 1	<0.001
10	VAR-LiNGAM	+0.518 [+0.477, +0.558]	40 / 0	<0.001
20	PCMCI+	+0.052 [-0.001, +0.105]	20 / 20	0.117
20	Rhino+g	+0.072 [+0.009, +0.134]	26 / 14	0.117
20	DYNOTEARS	+0.133 [+0.054, +0.212]	27 / 13	0.017
20	Rhino+s	+0.235 [+0.194, +0.276]	39 / 1	<0.001
20	VAR-LiNGAM	+0.412 [+0.359, +0.464]	40 / 0	<0.001

What happened. Rhino has the highest mean temporal F1 at every dimension (Table 5). It beats DYNOTEARS, VAR-LiNGAM, Rhino+g and Rhino+s on the same datasets at 5 and 10 nodes with Holm p below 0.01 (Table 6). It beats PCMCI+ at 5 and 10 nodes by 0.12 and 0.08. At 20 nodes the gain over PCMCI+ is 0.05 with a bootstrap interval of 0.001 to 0.103. Rhino wins on 20 datasets and loses on 20. The Holm p value is 0.117. I cannot call this a difference. VAR-LiNGAM is close to zero everywhere.

The instantaneous part is the hardest. Rhino reaches about 0.25 and the three baselines stay near 0.1 or below.

Was it expected. The paper reports the same order and says that Rhino is best or competitive, especially in low dimensions [1, Section 6.1]. My result agrees with that wording. The paper does not say that the gain shrinks with the dimension. In my data it does. All methods lose F1 as the dimension grows, but Rhino loses most in absolute terms (0.67 to 0.44) and the spread over datasets grows (standard deviation 0.14 to 0.19). A larger graph gives the model more to fit with the same number of series, and the variance of the training result grows with it.

Limit. Differences of this size depend on the data generator. A different mix of graph types or noise would change them. I did not run 40 nodes. So I cannot say whether the gain over PCMCI+ stays at zero, shrinks further or reverses.

5.2 Accuracy and cost

Table 7. Median wall time per dataset in minutes (one worker, 2 threads)

Method	5 nodes	10 nodes	20 nodes
Rhino	26.8	48.8	169.0
Rhino+g	12.9	21.1	91.6
Rhino+s	37.9	28.4	86.3
DYNOTEARS	0.32	0.35	0.79
VAR-LiNGAM	0.30	0.39	0.59
PCMCI+	0.31	0.39	1.02

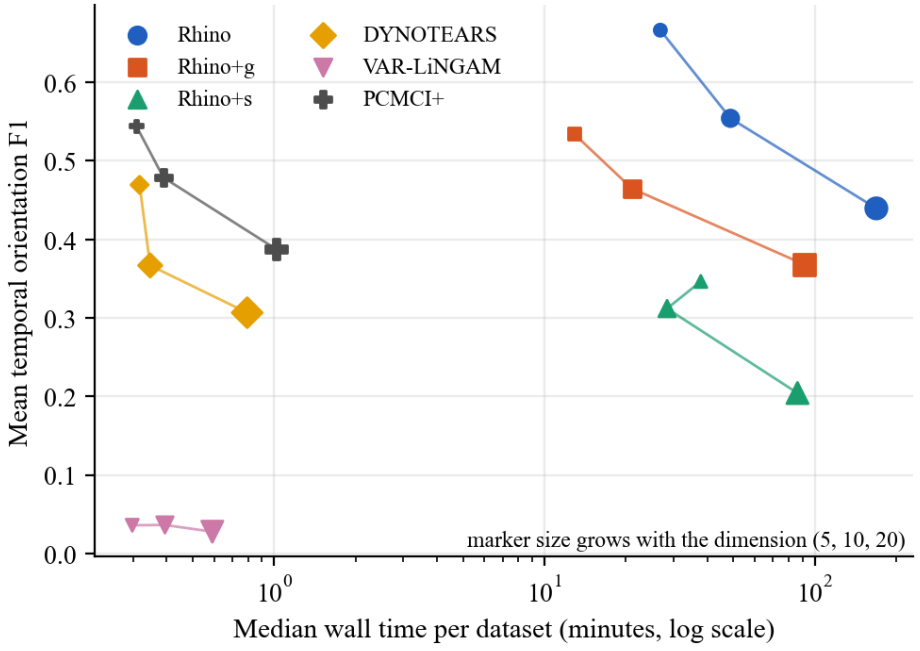


Fig. 2: Accuracy against cost. Each marker is one method at one dimension. The horizontal axis is logarithmic. PCMCI+, DYNOTEARS and VAR-LiNGAM are about 100 to 200 times cheaper than Rhino.

The three Rhino variants take 13 to 38 minutes per dataset at 5 nodes and 86 to 169 minutes at 20 nodes. The baselines need about one minute or less. So Rhino buys its gain at 5 and 10 nodes with a cost of two orders of magnitude. At 20 nodes it buys nothing measurable over PCMCI+ in F1, and PCMCI+ is cheaper by a factor of about 170. This is the clearest answer to the question of when a flexible model is worth its cost. It is worth it when the data have history-dependent noise (Section 5.3). PCMCI+ has a long tail. Its slowest job at 20 nodes took 90 minutes. This matches the paper's remark that PCMCI+ becomes expensive in high dimensions.

5.3 Ablations: history-dependent noise, instantaneous effects and graph type

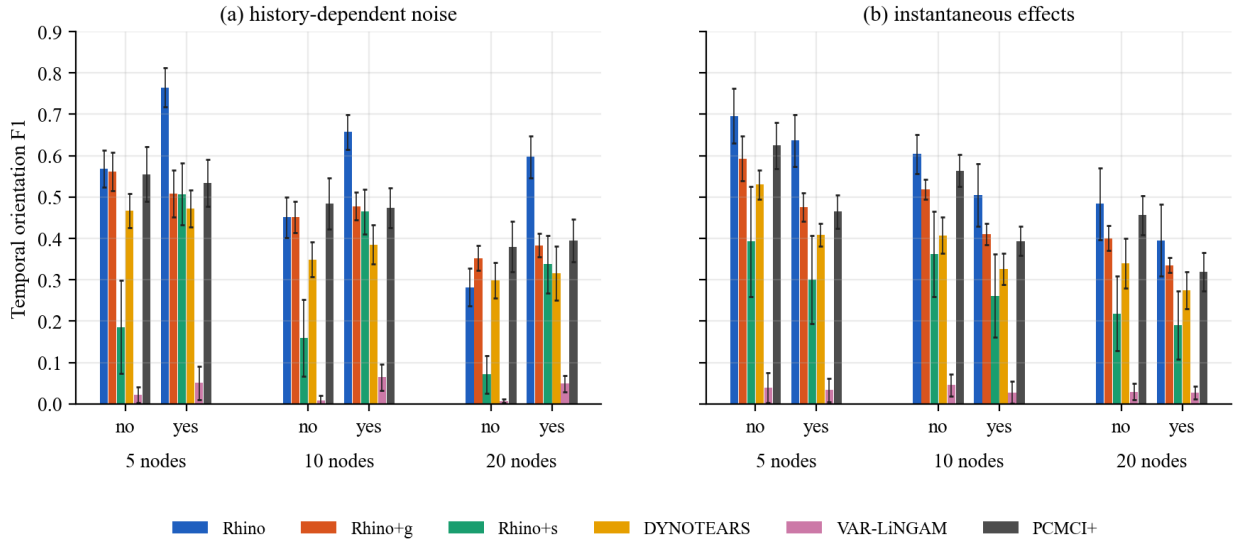


Fig. 3: Temporal orientation F1 split by history-dependent noise (a) and by instantaneous effects (b). Means over 20 datasets per bar with the 95 % interval.

Table 8. Rhino minus Rhino+g and PCMCI+ in temporal F1, split by history-dependent noise (20 datasets each)

Nodes	History-dep. noise	Other method	Mean difference [95 % interval]	Wins / losses	p (Holm)
5	yes	Rhino+g	+0.258 [+0.200, +0.314]	19 / 1	<0.001
5	yes	PCMCI+	+0.231 [+0.165, +0.297]	18 / 2	<0.001
5	no	Rhino+g	+0.006 [-0.029, +0.040]	10 / 10	1.000
5	no	PCMCI+	+0.013 [-0.028, +0.052]	10 / 10	1.000
10	yes	Rhino+g	+0.180 [+0.143, +0.219]	19 / 1	<0.001
10	yes	PCMCI+	+0.184 [+0.147, +0.221]	19 / 1	<0.001
10	no	Rhino+g	-0.001 [-0.025, +0.025]	9 / 11	1.000
10	no	PCMCI+	-0.033 [-0.064, -0.003]	7 / 13	0.291
20	yes	Rhino+g	+0.214 [+0.152, +0.274]	19 / 1	<0.001
20	yes	PCMCI+	+0.202 [+0.169, +0.235]	20 / 0	<0.001
20	no	Rhino+g	-0.071 [-0.119, -0.018]	7 / 13	0.129
20	no	PCMCI+	-0.098 [-0.122, -0.074]	0 / 20	<0.001

What happened. With history-dependent noise Rhino is better than Rhino+g by 0.26, 0.18 and 0.21 at 5, 10 and 20 nodes and better than PCMCI+ by 0.23, 0.18 and 0.20. All six Holm p values are below 0.001 (Table 8). Without history-dependent noise the picture changes. At 5 and 10 nodes Rhino and Rhino+g are equal (difference 0.006 and −0.001). Rhino is 0.098 below PCMCI+ at 20 nodes. It loses on all 20 datasets (Holm p below 0.001). Rhino is also 0.071 below Rhino+g at 20 nodes, but this difference is not significant after correction (p = 0.13).

Reading. This supports H1 and H2. The gain of Rhino comes from the history-dependent noise and from nothing else. When the data do not have it, Rhino+g fits as well at small size and PCMCI+ is better at 20 nodes. The reason is probably the cost of flexibility. Without history dependence the conditional flow adds parameters that the data cannot constrain. I did not test this reason. It follows from the pattern but I have no experiment that isolates it.

Instantaneous effects. All the main methods score lower on data with instantaneous effects (Fig. 3b). The drop is 0.06 to 0.17 in temporal F1. The paper calls all methods robust to this setting [1, Appendix F.3]. In my data the drop is clear but Rhino stays the best or close to the best. Rhino has the higher mean than Rhino+g and PCMCI+ in all six cases. After Holm correction over these twelve comparisons only three are significant. They are both methods at 5 nodes with instantaneous effects (p = 0.001) and Rhino+g at 10 nodes without them (p = 0.023). The sample is only 20 datasets for each bar. The

instantaneous F1 itself is low for every method (Fig. 1). So H3 is only partly tested here. I did not run Rhino+NoInst on synthetic data. I did run it on Netsim (Section 7).

Graph type. Rhino, Rhino+s, PCMCI+ and VAR-LiNGAM do better on Erdős-Rényi graphs than on scale-free graphs at every dimension. For example at 20 nodes Rhino gets 0.515 on ER and 0.364 on SF, and PCMCI+ gets 0.474 and 0.302. DYNOTEARS does better on SF at 10 and 20 nodes (0.204 on ER and 0.409 on SF at 20 nodes). Rhino+g does so too (0.328 and 0.408 at 20 nodes). The paper reports DYNOTEARS as the only exception [1, Appendix F.3]. So I reproduce the rule with one more exception. The numbers are in Results/tables/synth_factors_orientation.csv.

5.4 Lag ablation (Table 1 of the paper)

Table 9. Lag ablation. Temporal F1 and training log-likelihood, this run against the paper (mean ± standard deviation)

Nodes	Model lag	F1, this run	F1, paper	Log-lik., this run	Log-lik., paper
5	1	0.42 ± 0.10	0.34 ± 0.12	−3.67 ± 1.67	−4.14 ± 1.63
5	2	0.67 ± 0.14	0.59 ± 0.20	−3.25 ± 1.67	−3.83 ± 1.62
5	3	0.59 ± 0.16	0.56 ± 0.22	−3.11 ± 1.66	−3.75 ± 1.64
10	1	0.36 ± 0.09	0.28 ± 0.11	−7.21 ± 2.15	−7.97 ± 2.09
10	2	0.55 ± 0.14	0.49 ± 0.18	−6.52 ± 2.20	−7.21 ± 2.22
10	3	0.50 ± 0.14	0.45 ± 0.20	−6.48 ± 2.29	−7.01 ± 1.91
20	1	0.29 ± 0.11	0.25 ± 0.13	−14.56 ± 3.04	−15.62 ± 3.16
20	2	0.44 ± 0.19	0.39 ± 0.22	−13.54 ± 2.75	−14.70 ± 2.87
20	3	0.40 ± 0.18	0.37 ± 0.21	−13.64 ± 2.87	−14.72 ± 2.82

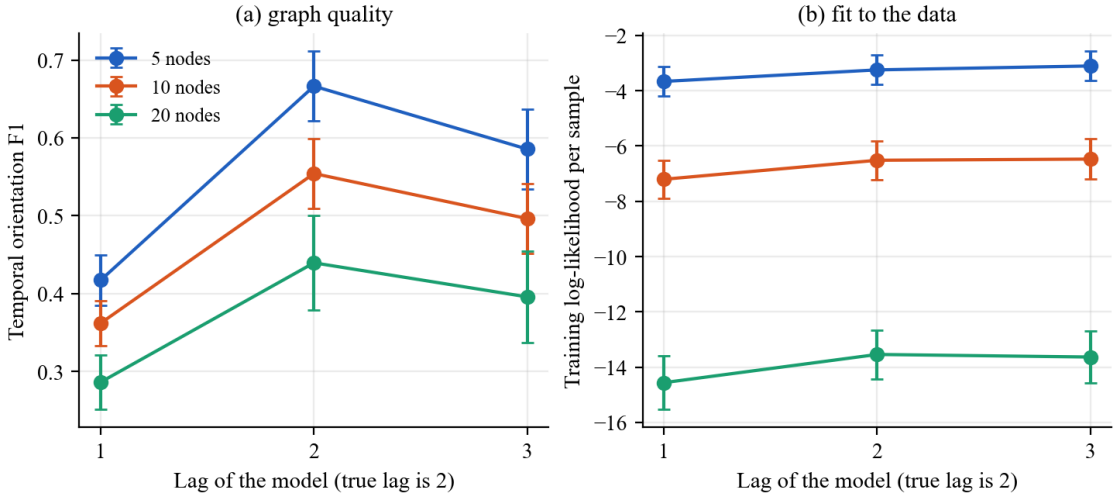


Fig. 4: Lag ablation. Means over 39 to 40 datasets with the 95 % interval. The true lag is 2. The log-likelihood is a training value.

What happened. A model lag of 2 gives the best F1 at every dimension (Table 9). Lag 1 is worse by 0.25, 0.19 and 0.15 (Holm p below 0.001). Lag 3 is worse than lag 2 by 0.08, 0.06 and 0.05 (also p below 0.001). The training log-likelihood rises from lag 1 to lag 2 at every dimension (paired difference 0.42, 0.69 and 1.02, p below 0.001). From lag 2 to lag 3 it changes by less than the spread, except at 5 nodes where it rises by 0.14 (Holm p = 0.026).

Reading. This supports H4 for lag 1 and shows that lag 3 is not free. The paper concludes that Rhino is robust to a longer lag. I see the same shape but with a measurable cost of 0.05 to 0.08 F1. The extra lag adds possible edges and the F1 drops while the likelihood does not improve. The absolute values are 0.05 to 0.08 higher than the paper for lag 2. My datasets are not the same as the paper's, because the grid of the paper is not fully specified (Section 11). So I compare the pattern and not the absolute numbers. The log-likelihood is a training value, so it cannot show overfitting.

5.5 Sparsity weight

The paper uses a sparsity weight of 9 for Rhino. The released config has 5. I ran both at 5 and 10 nodes (40 datasets each). The paired difference in temporal F1 is +0.014 at 5 nodes (bootstrap interval [-0.009, +0.036], p = 0.15) and +0.002 at 10 nodes ([-0.019, +0.024], p = 0.87). So the choice between 9 and 5 does not matter in this range. The open point between the paper and the code has no effect on my conclusions.

5.6 Error analysis: what kind of mistakes

Table 10. Oriented precision, oriented recall, predicted edges relative to true edges and the gap between adjacency F1 and orientation F1

Nodes	Method	Precision	Recall	Predicted / true edges	Adjacency F1 minus orientation F1
5	Rhino	0.74	0.62	1.04	0.007
5	Rhino+g	0.60	0.50	1.00	0.034
5	Rhino+s	0.63	0.26	0.40	0.005
5	DYNOTEARS	0.41	0.59	2.08	0.098
5	VAR-LiNGAM	0.20	0.02	0.04	0.002
5	PCMCI+	0.89	0.40	0.54	0.007
10	Rhino	0.68	0.49	1.30	0.006
10	Rhino+g	0.51	0.46	1.64	0.027
10	Rhino+s	0.58	0.23	0.60	0.007
10	DYNOTEARS	0.31	0.52	3.76	0.083
10	VAR-LiNGAM	0.31	0.02	0.06	0.006
10	PCMCI+	0.80	0.36	0.80	0.011
20	Rhino	0.63	0.39	1.55	0.002
20	Rhino+g	0.42	0.42	2.88	0.017
20	Rhino+s	0.46	0.16	0.71	0.002
20	DYNOTEARS	0.28	0.48	6.40	0.065
20	VAR-LiNGAM	0.49	0.01	0.06	0.001
20	PCMCI+	0.69	0.30	1.09	0.012

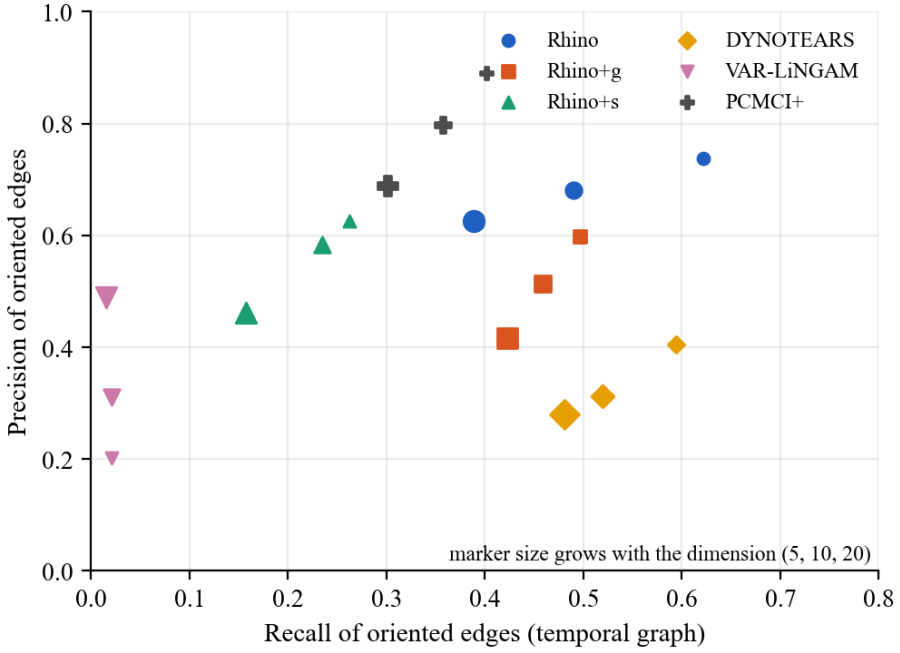


Fig. 5: Precision against recall of oriented edges in the temporal graph. Marker size grows with the dimension (5, 10, 20).

What happened. The methods fail in different ways. PCMCI+ has the highest precision (0.89, 0.80, 0.69) and the lowest recall among the useful methods (0.40, 0.36, 0.30). It predicts too few edges at 5 and

10 nodes. DYNOTEARS predicts two to six times too many edges and has precision between 0.28 and 0.41. VAR-LiNGAM predicts almost no edges (recall 0.02). Rhino sits in between. It predicts about the right number of edges at 5 nodes (ratio 1.04) and too many at 20 nodes (ratio 1.55). Rhino+g predicts too many at 10 and 20 nodes (1.64 and 2.88). Rhino+s predicts too few (0.40 to 0.71) and so has low recall.

Rhino rarely finds a pair of variables and gets the direction wrong. The gap between adjacency F1 and orientation F1 is 0.002 to 0.007 for Rhino and 0.065 to 0.098 for DYNOTEARS. So the main mistakes of Rhino are missing edges and, at larger size, extra edges, and not wrong directions. Rhino+s is the weakest variant and does worse than Rhino+g. The paper finds Rhino+s close to Rhino. I do not reproduce this (Section 9). I did not test the reason. One possible reason is that a spline noise with no history dependence is hard to fit and the model then keeps too few edges. The low edge count is consistent with that, but it is not a test.

Failure case of Rhino. The clearest failure is data without history-dependent noise at 20 nodes. There Rhino is below PCMCI+ on every one of the 20 datasets. A user who does not know whether the noise depends on the history should run Rhino+g or PCMCI+ as well.

6. Treatment Effect Estimation

Table 11. RMSE of the average treatment effect over 40 datasets (lower is better)

Nodes	Method	E[ATE], mean [95 % interval]	E[ATE], median	ML ATE, mean	Median time (min)
5	Rhino	0.866 [0.604, 1.128]	0.496	0.865	27
5	Rhino+g	0.816 [0.541, 1.092]	0.509	0.821	13
5	Rhino+s	0.567 [0.418, 0.716]	0.422	0.565	38
10	Rhino	0.766 [0.579, 0.954]	0.553	0.762	49
10	Rhino+g	0.568 [0.452, 0.683]	0.409	0.584	21
10	Rhino+s	0.523 [0.406, 0.640]	0.411	0.519	28
20	Rhino	0.484 [0.351, 0.617]	0.431	0.479	169
20	Rhino+g	0.737 [0.579, 0.896]	0.624	0.741	92
20	Rhino+s	0.338 [0.271, 0.406]	0.276	0.332	86

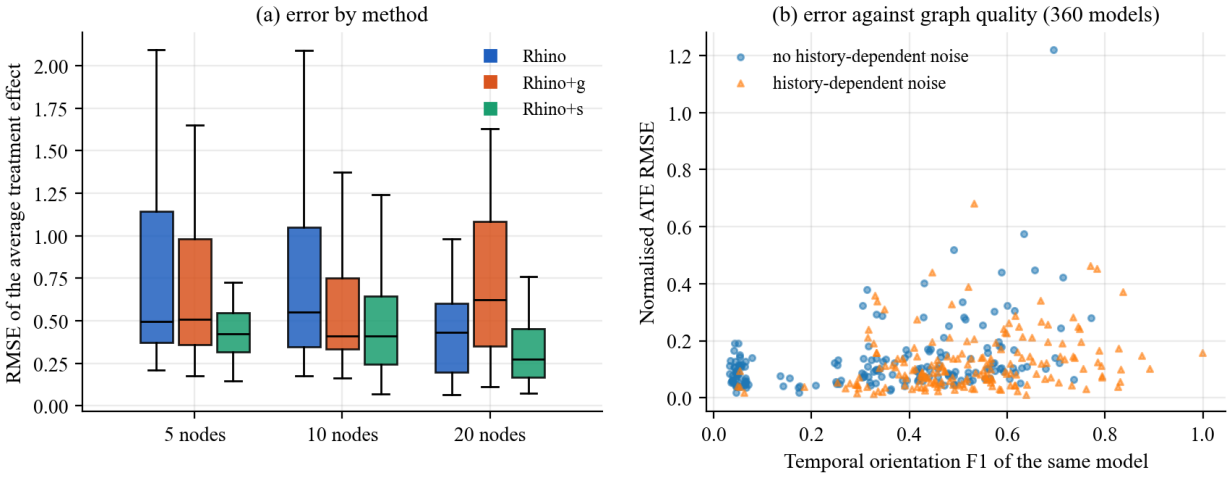


Fig. 6: (a) RMSE of the average treatment effect by method. (b) The same error, normalised, against the temporal F1 of the same model. Each point is one trained model.

What happened. Rhino+s has the lowest mean error at every dimension (0.567, 0.523 and 0.338), although it has the worst graph F1. The paired tests are weak at 5 and 10 nodes. All Holm p values there are above 0.06. At 20 nodes the error of Rhino is 0.25 lower than that of Rhino+g ($p = 0.002$) and the error of Rhino+s is 0.40 lower than that of Rhino+g (p below 0.001). E[ATE] and ML ATE agree within 0.02 in every row, which suggests that the posterior over graphs is concentrated.

Is the treatment effect linked to graph quality. Over all 360 models the Spearman correlation between F1 and the error is +0.44. It is positive, so a better graph goes with a larger error. This comes mainly from the type of data. Datasets with history-dependent noise give both higher F1 and larger effects. Inside groups of equal method, dimension, history dependence and instantaneous setting, the correlation falls to +0.15. So I find no sign that a better graph gives a better treatment effect. The paper makes the same remark and calls it a question for further study [1, Appendix D]. I only confirm it. I do not explain it. A possible reason is that the effect depends on the few paths between the treated and the target variable and not on the whole graph.

7. Results on Real Data

7.1 DREAM3

Table 12. DREAM3 AUROC of the summary graph. This run, mean ± standard error over runs (n), paper value in brackets

Method	E. coli 1	E. coli 2	Yeast 1	Yeast 2	Yeast 3
cMLP	0.644	0.568	0.585	0.506	0.528
cLSTM	0.629	0.609	0.579	0.519	0.555
TCDF	0.614	0.647	0.581	0.556	0.557
SRU	0.657	0.666	0.617	0.575	0.550
eSRU	0.660	0.629	0.627	0.557	0.550
DYNOTEARS	0.569	0.547	0.523	0.530	0.507
PCMCI+	0.540	0.532	0.522	0.505	0.515
Rhino+g	0.643 ± 0.006 (n=5) [0.673]	0.645 ± 0.007 (n=5) [0.665]	0.629 ± 0.006 (n=5) [0.659]	0.585 ± 0.002 (n=2) [0.598]	0.565 ± 0.004 (n=5) [0.588]
Rhino	0.657 ± 0.010 (n=5) [0.685]	0.656 ± 0.012 (n=2) [0.680]	0.634 ± 0.012 (n=5) [0.664]	0.553 ± 0.000 (n=2) [0.585]	0.554 ± 0.008 (n=5) [0.567]

cMLP, cLSTM, TCDF, SRU and eSRU are single-run numbers that the paper quotes from [5]. DYNOTEARS and PCMCI+ are single runs. Rhino and Rhino+g use the hyperparameters printed in the paper.

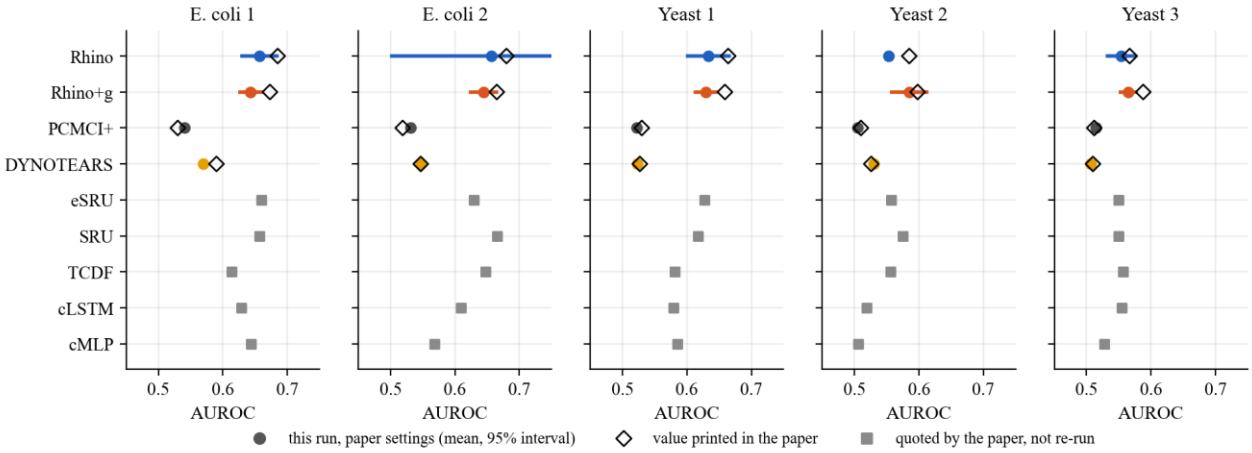


Fig. 7: DREAM3 AUROC. Coloured dots are this run with the 95 % interval, diamonds are the paper's values and grey squares are numbers quoted by the paper.

What happened. Rhino and Rhino+g are clearly above DYNOTEARS and PCMCI+ on all five networks. For E. coli 1 the 95 % interval of Rhino (0.630 to 0.683) does not reach the values of DYNOTEARS (0.569) or PCMCI+ (0.540). Against the quoted Granger methods the picture is mixed. Rhino ties SRU and eSRU on E. coli 1 (0.657 against 0.657 and 0.660). It is slightly below SRU on E. coli 2 (0.656 against 0.666). It is above the best Granger method on Yeast 1 (0.634 against 0.627). On Yeast 2 and Yeast 3 Rhino+g is the best method (0.585 against 0.575, and 0.565 against 0.557) and Rhino itself is below them. These gaps are 0.003 to 0.010. The quoted numbers have no interval. So I cannot call any of these gaps real.

Against the paper. On the networks with five runs my values are 0.01 to 0.03 below the paper's for both Rhino and Rhino+g (for example Rhino on E. coli 1: 0.657 against 0.685, with standard error 0.010 against 0.003). So I do not reproduce the printed numbers. I reproduce the ranking of the baselines and the pattern between Rhino and Rhino+g. Rhino is better on E. coli 1, E. coli 2 and Yeast 1. Rhino+g is better on Yeast 2 and Yeast 3. The paper has the same split. None of these differences is significant (Welch p between 0.04 and 0.76, and 0.20 or higher after Holm). I did not find the reason for the gap. Possible reasons are my train and validation split (assumption in Data/README.md), the hyperparameter tuning of the paper which I did not repeat, and different data preprocessing. None is tested.

Limits. E. coli 2 and Yeast 2 with Rhino and Yeast 2 with Rhino+g have two runs and not five, because the other runs hit the time limit. The intervals for these cells are very wide (Fig. 7). The orientation F1 and SHD of the paper (Tables 6 and 7 of [1]) are not reproduced. The paper tunes the rounding threshold for these two metrics. I used the fixed value 0.5. At this threshold Rhino predicts many more edges than the truth and the values are not comparable. I therefore do not report them as a result. A run of the released configs on DREAM3 did not finish within the time budget.

7.2 Netsim

Table 13. Netsim. This run (mean ± standard error over five runs) next to the paper (Tables 3 and 8)

Method and setting	AUROC, this run	AUROC, paper	Orient. F1, this run	Orient. F1, paper	SHD, this run	SHD, paper
Rhino, paper steps	0.987 ± 0.002	0.99 ± 0.001	0.287 ± 0.013	0.551 ± 0.048	40.7 ± 0.7	13.8 ± 1.5
Rhino, released config	0.931 ± 0.013	0.99 ± 0.001	0.543 ± 0.018	0.551 ± 0.048	15.6 ± 1.0	13.8 ± 1.5
Rhino+g, paper steps	0.972 ± 0.004	0.974 ± 0.002	0.230 ± 0.012	0.539 ± 0.036	38.8 ± 0.8	10.4 ± 1.08
Rhino+g, released config	0.905 ± 0.013	0.974 ± 0.002	0.458 ± 0.055	0.539 ± 0.036	13.0 ± 1.3	10.4 ± 1.08
Rhino+NoInst, paper steps	0.912 ± 0.009	0.93 ± 0.006	0.160 ± 0.006	0.212 ± 0.014	45.6 ± 0.9	29 ± 1.5
Rhino+NoInst, released config	0.857 ± 0.008	0.93 ± 0.006	0.193 ± 0.014	0.212 ± 0.014	31.5 ± 2.0	29 ± 1.5
DYNOTEARS	0.896	0.90	0.686	0.341	6	17
PCMCI+	0.830	0.83 ± 0	0.410	0.41	18	18
VAR-LiNGAM	0.836	0.84 ± 0	0.296	0.44	29	18

Paper steps are the 5 augmented Lagrangian steps printed in the paper. The released config trains for many more steps. The paper gives one value per model in each table and does not say how many steps belong to which table.

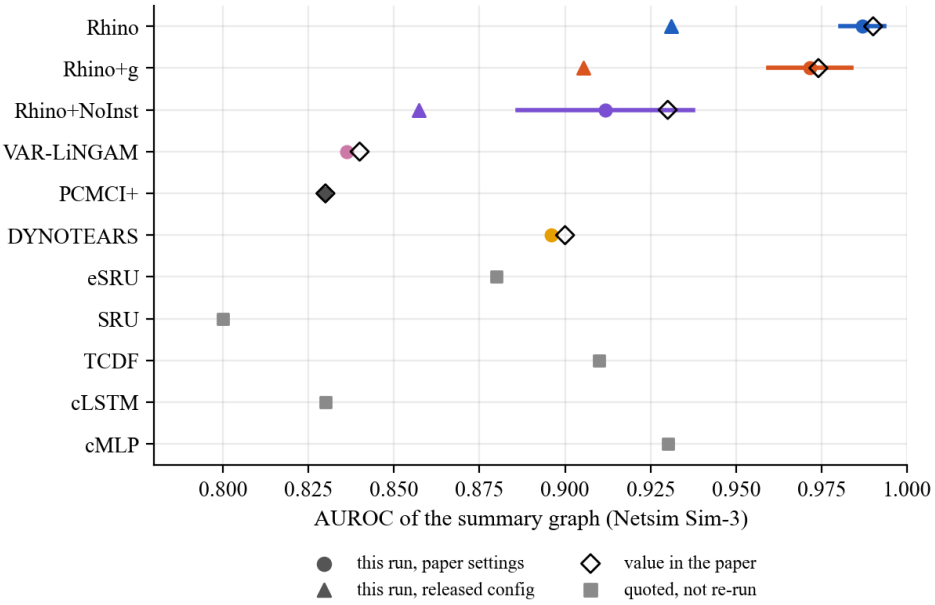


Fig. 8: Netsim AUROC of the summary graph. Coloured dots are this run with the 95 % interval, triangles are the released config, diamonds are the paper's values and grey squares are quoted by the paper.

What happened. With the settings printed in the paper (5 augmented Lagrangian steps) the AUROC of Rhino is 0.987 (paper 0.990). Rhino+g reaches 0.972 (paper 0.974) and Rhino+NoInst reaches 0.912

(paper 0.93). All three paper values lie inside my 95 % intervals. The baselines also match the paper: DYNOTEARS 0.896, PCMCI+ 0.830 and VAR-LiNGAM 0.836. This is the best reproduction in my study.

Ablations. Removing the instantaneous effects costs 0.075 AUROC (Welch p below 0.001). This supports H3 on real data. Replacing the history-dependent noise by Gaussian noise costs 0.015 (p = 0.020, uncorrected). The paper also finds this effect to be small. With five runs per cell I regard the evidence as suggestive and not strong.

Training length. With the released config, which trains for many more steps, the AUROC is lower: 0.931, 0.905 and 0.857. The loss against the paper steps is 0.056, 0.066 and 0.055 (Welch p between 0.002 and 0.012). The F1 and SHD go the other way. The released config gives F1 0.543 and SHD 15.6 for Rhino, which is close to the paper's 0.551 and 13.8. The paper steps give F1 0.287 and SHD 40.7. So the paper's AUROC (Table 3) and its F1 and SHD (Table 8) match two different training lengths in my runs. The paper does not say this. This is my reading of the numbers. I did not confirm it with the authors.

Where my numbers differ. DYNOTEARS reaches F1 0.686 and SHD 6 in my run, but 0.341 and 17 in the paper. VAR-LiNGAM gets 0.296 and 29, against 0.44 and 18. PCMCI+ matches (0.410 and 18). I did not find the reason for DYNOTEARS. A different threshold for its weighted matrix may be the source, but I did not test it.

7.3 Training curves and probability quality

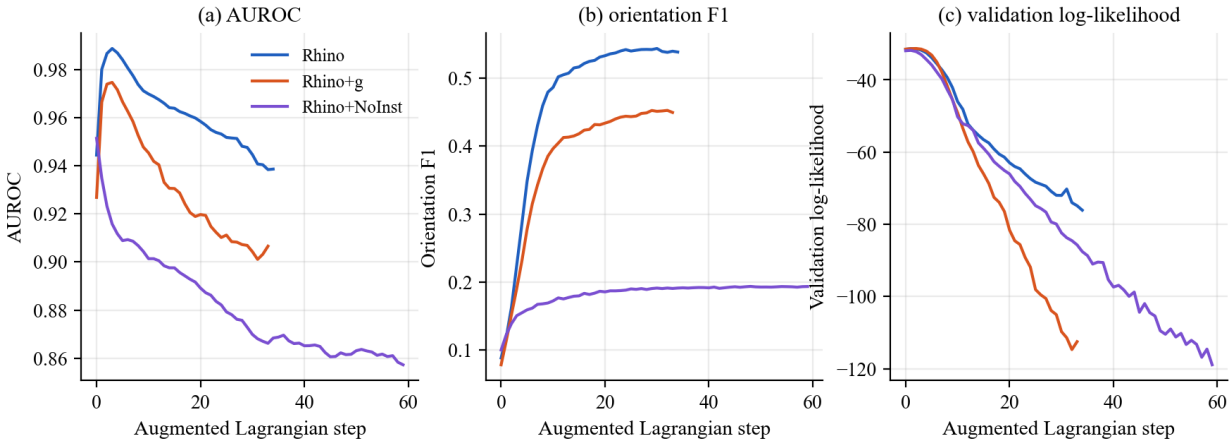


Fig. 9: Netsim training curves. Mean over five runs of the AUROC (a), the orientation F1 (b) and the validation log-likelihood (c) after each augmented Lagrangian step, with the released configs.

What happened. For Rhino the AUROC peaks at step 3 with 0.989 and then falls to 0.939 at step 34. The F1 rises from 0.088 to 0.538. The validation log-likelihood falls from -31.5 to -76.1. Rhino+g and Rhino+NoInst show the same pattern (AUROC 0.975 to 0.907, and 0.951 to 0.857). The best validation log-likelihood comes at step 1 for Rhino.

Reading. This is the pattern that the paper reports in its Fig. 8 and explains as overfitting. As training goes on, the edge probabilities move to 0 or 1. The rounded graph improves because more edges cross 0.5 in the right way. The ranking of the edges gets worse, because wrong edges also receive probabilities close to 1. The model becomes more confident and not more correct. The validation log-likelihood falls together with the AUROC, so it is a usable signal to stop training. For Rhino it would have chosen step 1, where the AUROC is 0.980. This is evidence on probability quality, but it is indirect.

Probability quality: what I did and did not measure

A graph model should state how sure it is. Rhino does this through q(G). I measured three things about it: the AUROC of the edge probabilities, the training log-likelihood (Table 9) and the validation log-likelihood on Netsim (Fig. 9c). I did **not** measure the Brier score, the expected calibration error or a

reliability diagram of the edge probabilities. The edge probability matrices were not part of the exported results, so I cannot compute them from the data I have. I list this as a gap and plan it for M4 (Section 13). I also did not report a held-out log-likelihood for the synthetic runs. The values in Table 9 are training values.

8. Robustness

I studied robustness along five axes: the dimension (Sections 5.1 and 5.2), the graph type and the data mismatch with the model (Section 5.3), the model lag (Section 5.4), the sparsity weight (Section 5.5) and the training length (Section 7.2). The results in short are these.

- **Dimension.** F1 falls for every method. The gain of Rhino over PCMCI+ falls from 0.12 to 0.05 and stops being significant at 20 nodes. The spread of Rhino grows.
- **Mismatch of the data with the model.** Rhino is the best method when the noise is history-dependent. It is not the best when the noise is not history-dependent and the graph is large (Table 8).
- **Model lag.** A lag that is too small loses 0.15 to 0.25 F1. A lag that is too large loses 0.05 to 0.08.
- **Sparsity weight.** 9 against 5 changes nothing measurable.
- **Training length.** It changes AUROC and F1 in opposite directions. The right stopping rule depends on the metric (Section 7.3).
- **Not tested.** Sample size (the number of series was fixed at 50), noise level, missing data and the graph density beyond the two families. I did not run these.

9. What Was Reproduced and What Was Not

Table 14. Claims of the paper against my evidence

Claim of the paper	My evidence	Status
Rhino is best or competitive on synthetic data	Highest mean F1 at 5, 10, 20 nodes. Significant against PCMCI+ at 5 and 10 nodes only.	Partly reproduced
History-dependent noise helps	Rhino above Rhino+g by 0.18 to 0.26 with such noise. No gain without it.	Reproduced
Rhino+s is close to Rhino	Rhino+s is 0.23 to 0.32 below Rhino in paired F1.	Not reproduced
Rhino is robust to a longer lag, a shorter lag hurts (Table 1)	Same pattern. A longer lag costs 0.05 to 0.08 F1. Absolute values differ.	Reproduced in shape
Most methods do better on ER than SF, DYNOTEARS is the exception	Same for four methods. DYNOTEARS and also Rhino+g do better on SF at 10 and 20 nodes.	Mostly reproduced
Rhino and Rhino+g beat the other methods on DREAM3 (Table 2)	Above DYNOTEARS and PCMCI+. About equal to the best Granger method. Values 0.01 to 0.03 below the paper.	Partly reproduced
Netsim AUROC 0.99 for Rhino (Table 3)	0.987 with the paper steps. Paper value inside my interval.	Reproduced
Instantaneous effects matter on Netsim	AUROC 0.987 against 0.912 without them.	Reproduced
AUROC and validation likelihood fall while F1 rises (Fig 8)	Same pattern in three settings.	Reproduced
Treatment effect error is little related to discovery quality (Fig 2)	No sign of a link. Weak positive correlation inside groups (0.15).	Consistent
DREAM3 orientation F1 and SHD (Tables 6 and 7)	Not comparable. The paper tunes the threshold.	Not reproduced
Results at 40 nodes, PCMCI+ over one week at 40 nodes	Not run.	Not tested

10. Revised SOTA Position

I follow the loop of the course: identify, understand, compare, validate, reposition.

- **Identify.** The anchor is the set of methods in the base paper. The synthetic comparator is PCMCI+. The real-data comparators are the nonlinear Granger methods.
- **Understand.** Rhino models nonlinearity, instantaneous effects and history-dependent noise together. Table 1 shows that no other method does all three.

- **Compare.** Same data and same conditions for all methods, paired tests, five or more runs on real data.
- **Validate.** Most claims hold in shape. Several do not hold in size (DREAM3 values, Rhino+s). The 40-node results are untested.
- **Reposition.** The statement below.

Rhino is the strongest of the six methods I ran when the data have history-dependent noise. This holds at 5, 10 and 20 nodes. Over all datasets of 20 nodes its advantage over PCMCI+ in F1 cannot be separated from zero, and it costs about 170 times more time. When the noise does not depend on history, Rhino has no advantage over its Gaussian variant or over PCMCI+, and PCMCI+ is better at 20 nodes. On real data it is clearly better than the linear baselines and about level with the best Granger methods. The gaps there are small and the Granger numbers have no interval. So my position is narrower than the claim of the paper. Rhino helps in a well defined region and it is not a general replacement. I did not review work after the paper, so I do not claim a position against newer methods.

11. Limits and Open Points

- **No 40-node results.** They were too expensive on the available machine. Table 1, Fig 1 and Fig 2 of the paper are therefore only checked up to 20 nodes.
- **The synthetic grid is my reading of the paper.** The paper says 160 datasets. The released generator writes 480. I used connection factor 2 and the same graph family for both graphs. I could not check this against the authors.
- **One model seed per synthetic dataset.** The intervals show spread over datasets and not over training seeds.
- **DREAM3 is incomplete.** Three cells have two runs and not five. The released configs did not finish. Orientation F1 and SHD are not reproduced.
- **Quoted baselines.** cMLP, cLSTM, TCDF, SRU and eSRU are not re-run. They have no uncertainty.
- **Probability quality.** No calibration measures (Section 7.3).
- **Assumptions not stated in the paper.** F1 on synthetic data (I report orientation and adjacency F1, both in the files), the Netsim subject numbering, the DREAM3 split and the Netsim lag. They are listed in Results/STATUS.md.
- **Problems in the released code.** The treatment effect flag needs a workaround, and the evaluation of the diagonal needs a flag on Netsim. Both are in Results/STATUS.md. Five early jobs failed because of a race in the logging library. They were run again.
- **SHD.** The SHD of the released code is not always equal to the sum of missed and extra edges. I did not trace why. I report it in the files and base my conclusions on F1.
- **Newer methods.** I did not survey work after the paper.

12. Revisiting M1 and M2

- **Problem and research question.** The research question is fixed in Section 1. M1 and M2 use the same wording.
- **Baselines.** The baseline set is DYNOTEARS, VAR-LiNGAM and PCMCI+, with the nonlinear Granger numbers quoted from the paper.
- **KPIs.** Orientation F1, adjacency F1, precision, recall, SHD, AUROC, treatment effect RMSE and run time. Calibration is planned for M4.
- **Assumptions.** Causal stationarity, causal sufficiency, a graph that is acyclic at time t, and the generator settings of Section 4.
- **Setup.** The code of the authors is used without edits. The setup was checked on a small case before the full runs.

13. Reproducibility and Plan for M4

All results come from the scripts in Code/scripts, which write the tables and figures in Results/. Nothing is typed by hand. Code/README.md gives the commands, the environment and the seeds. Code/environment/pip_freeze.txt lists the exact package versions. Each job has its own result file in Results/raw. Jobs that did not finish are in Results/failed_infra with their logs. The authors' code is not copied here. It is used at tag v0.0.0 without edits [15]. The extra statistics of this report (paired tests by group, bootstrap intervals, the precision and recall table, the treatment effect correlations and the Welch tests) are computed from Results/raw by Code/scripts/report_extra_stats.py and Code/scripts/report_real_stats.py.

Table 15. Plan for M4 (may change as I learn more)

Item	Planned work
Calibration	Re-run a subset, save the edge probabilities and report Brier score, expected calibration error and reliability diagrams.
Sample size	Vary the number of series at 5 and 10 nodes and plot SHD and F1 against the sample size.
Missing runs	The two missing DREAM3 seeds per cell, and the 40-node case if the budget allows.
Robustness	Noise level and missing values for the best three methods.
SOTA	A short check of work after the paper and the final position.

References

[1] W. Gong, J. Jennings, C. Zhang, and N. Pawlowski, “Rhino: Deep Causal Temporal Relationship Learning with History-dependent Noise,” in International Conference on Learning Representations (ICLR), 2023, arXiv:2210.14706.

[2] A. Hyvärinen, K. Zhang, S. Shimizu, and P. O. Hoyer, “Estimation of a structural vector autoregression model using non-Gaussianity,” Journal of Machine Learning Research, vol. 11, no. 5, 2010.

[3] R. Pamfil et al., “DYNOTEARS: Structure learning from time-series data,” in International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR, pp. 1595-1605, 2020.

[4] J. Runge, “Discovering contemporaneous and lagged causal relations in autocorrelated nonlinear time series datasets,” in Conference on Uncertainty in Artificial Intelligence (UAI), PMLR, pp. 1388-1397, 2020.

[5] S. Khanna and V. Y. F. Tan, “Economy statistical recurrent units for inferring nonlinear Granger causality,” arXiv:1911.09879, 2019.

[6] A. Tank, I. Covert, N. Foti, A. Shojaie, and E. Fox, “Neural Granger causality for nonlinear time series,” stat, vol. 1050, p. 16, 2018.

[7] M. Nauta, D. Bucur, and C. Seifert, “Causal discovery with attention-based convolutional neural networks,” Machine Learning and Knowledge Extraction, vol. 1, no. 1, p. 19, 2019.

[8] R. J. Prill et al., “Towards a rigorous assessment of systems biology models: the DREAM3 challenges,” PLoS ONE, vol. 5, no. 2, e9202, 2010.

[9] D. Marbach, T. Schaffter, C. Mattiussi, and D. Floreano, “Generating realistic in silico gene networks for performance assessment of reverse engineering methods,” Journal of Computational Biology, vol. 16, no. 2, pp. 229-239, 2009.

[10] T. Geffner et al., “Deep end-to-end causal inference,” arXiv:2202.02195, 2022.

[11] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, “DAGs with NO TEARS: Continuous optimization for structure learning,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.

[12] C. K. Assaad, E. Devijver, and E. Gaussier, “Survey and evaluation of causal discovery methods for time series,” Journal of Artificial Intelligence Research, vol. 73, pp. 767-819, 2022.

[13] S. Holm, “A simple sequentially rejective multiple test procedure,” Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65-70, 1979.

[14] F. Wilcoxon, “Individual comparisons by ranking methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, 1945.

[15] Microsoft, causica (Rhino code), release v0.0.0, github.com/microsoft/causica/tree/v0.0.0.